User registration and sessions

Module 5 - Lesson 2

Overview

Authentication proves who a user is. Registration stores a credential securely, and sessions carry that identity across requests without asking for the password again.

Key Concepts

- Never store plaintext passwords; hash them with bcrypt or argon2 and add a salt.
- Sessions come in two styles: opaque tokens stored server-side, or signed stateless tokens (JWT).
- Store the hashed session token or JWT in an httpOnly, secure cookie.
- Set sensible cookie flags: HttpOnly, Secure, SameSite, and an expiry.
- Provide account recovery and a logout path that invalidates the session.
- Rate-limit login and registration to slow down brute force and abuse.

Example

```
const hash = await bcrypt.hash(password, 12);
// store hash, not password
const token = jwt.sign({ sub: user.id }, SECRET, { expiresIn: '7d' });
res.cookie('session', token, { httpOnly: true, secure: true, sameSite: 'lax' });
```

Summary

Secure registration hashes passwords with salt. Sessions carry identity via httpOnly cookies or signed tokens, and careful cookie flags plus rate limiting protect the whole flow.

Practice Checklist

- Implement registration with hashed passwords and a login endpoint.
- Issue an httpOnly session cookie on login and verify it on a protected route.
- Add rate limiting to the auth endpoints.